PlasTrack
OBSERVATORIO SATELITAL

UNIDAD 1 · BI

UNIVERSIDAD TÉCNICA DE COTOPAXI
Facultad de Ciencias de la Ingeniería y Aplicadas
Carrera de Ingeniería en Sistemas de Información

MEMORIA DE CONSTRUCCIÓN

Memoria de construcción del dashboard «PlasTrack»

Documentación técnica del proceso, decisiones, pruebas y despliegue

AUTORES
Carlos · Gustavo · Gabriela · Daniela

ASIGNATURA
Inteligencia de Negocios · Unidad 1

DOCENTE
René Quisaguano

Latacunga — Ecuador

Noviembre de 2026

1. Resumen ejecutivo

Este documento explica cómo se construyó PlasTrack, el dashboard de inteligencia de negocios entregado como parte de la actividad cooperativa de la Unidad 1. Se describen los requisitos, el entorno técnico, los permisos utilizados, la estructura del proyecto, el proceso de implementación, las pruebas ejecutadas y el procedimiento de despliegue en GitHub Pages. El objetivo es que cualquier integrante del equipo —o el docente— pueda reproducir el resultado, auditar las decisiones y actualizar el tablero sin depender de conocimiento tácito.

El producto final es un sitio estático de una sola página con cinco KPIs, un mapa satelital interactivo (Leaflet), cuatro gráficos (Chart.js), una tabla filtrable y exportable, tema claro/oscuro y diseño responsivo. Consume 15 registros simulados de pasadas satelitales sobre Galápagos y se publica sin costo de infraestructura. El repositorio incluye el código, el dataset, las pruebas, la documentación y los scripts que generan este informe y el reporte técnico en PDF y DOCX.

Tabla 1. Ficha resumen del entregable

Elemento	Detalle
Nombre del producto	PlasTrack · Observatorio Satelital de Galápagos
Tipo	Dashboard web estático de inteligencia de negocios
Repositorio	https://github.com/carlos00012/plastrack
Dataset	data_copernicus_sample.json · 15 registros · 6.896 km² acumulados
Stack	HTML5, CSS3, JavaScript ES2022, Leaflet 1.9.4, Chart.js 4.4.3
Despliegue	GitHub Pages · HTTPS · URL pública
Documentación	Reporte técnico, memoria de construcción, README y bitácora de prompts
Licencia	MIT para el código; atribución para teselas y datos

2. Objetivo y alcance del documento

La memoria responde a tres preguntas: qué se construyó, cómo se construyó y cómo se verificó. No sustituye al reporte técnico, que contiene el benchmarking, la ficha de KPIs y la discusión académica. Su función es operativa: servir como guía de reproducción y como evidencia del trabajo realizado. Se documentan también los permisos y credenciales necesarios para publicar, con el fin de que el equipo pueda repetir el despliegue en su propia cuenta.

El alcance cubre el código del dashboard, el dataset, los scripts de documentación y el procedimiento de publicación. No cubre la integración con la API real de Copernicus, la autenticación de usuarios ni la operación de un servidor, que se describen como trabajo futuro en la sección 12.

3. Requisitos y trazabilidad

La rúbrica de la actividad se tradujo en requisitos verificables. La Tabla 2 muestra la relación entre cada requisito, la solución implementada y la evidencia que permite comprobarlo. Esta trazabilidad se usó como lista de verificación antes de la entrega.

Tabla 2. Matriz de trazabilidad requisito → solución → evidencia

ID	Requisito	Solución implementada	Evidencia
R01	Benchmarking de BI y librerías web	Matriz ponderada de 8 criterios y 4 alternativas	Reporte, Tablas 1–4
R02	Sustentación de la alternativa elegida	Análisis técnico y económico del stack web	Reporte, sección 4.5
R03	Ficha de KPIs	Ocho indicadores con fórmula, meta y frecuencia	Reporte, Tabla 7
R04	Bitácora de prompts de IA	Trece interacciones documentadas con ajustes	Reporte, Tabla 9
R05	Dashboard con tarjetas KPI	Cinco tarjetas dinámicas con delta y barra	index.html · #kpiGrid
R06	Mapa interactivo Leaflet	Teselas satelitales, marcadores, calor y zonas	assets/js/map.js
R07	Gráficos Chart.js	Línea dual, dona, barras y dispersión	assets/js/charts.js
R08	Consumo de data_copernicus_sample.json	fetch + validación de esquema	assets/js/analytics.js
R09	Diseño responsivo (móvil y PC)	Mobile-first con cinco puntos de quiebre	Capturas 390 y 1440 px
R10	Despliegue en GitHub Pages	Repositorio público + rama main	URL de producción
R11	Reporte en formato paper	DOCX y PDF con portada, secciones y anexos	docs/Reporte_Tecnico_PlasTrack.pdf y .docx
R12	Exposición y defensa individual	Guion de exposición y preguntas frecuentes	Sección 13 de esta memoria

4. Entorno técnico y permisos

4.1. Herramientas utilizadas

Tabla 3. Entorno de desarrollo

Herramienta	Versión	Uso
Windows	11 (10.0.26300)	Sistema operativo de desarrollo.
Node.js	v24.15.0	Servidor local, pruebas y scripts de documentación.
npm	11.12.1	Gestión de dependencias de desarrollo.
Git	2.55.0	Control de versiones.
GitHub CLI	2.102.0	Creación del repositorio y verificación de sesión.
Google Chrome	Última estable	Pruebas, capturas y generación de PDF.
Visual Studio Code	Última estable	Edición de código.

4.2. Permisos necesarios

La construcción local no requirió privilegios de administrador. Se necesitaron permisos de escritura en la carpeta del proyecto, acceso a la red para instalar dependencias y descargar teselas, y la capacidad de abrir un puerto local para servir el sitio. Para la publicación se utilizó una sesión de GitHub CLI ya autenticada en la cuenta del equipo, con los alcances repo y workflow, que son suficientes para crear el repositorio, subir el código y activar Pages.

Tabla 4. Permisos y alcances utilizados

Recurso	Permiso	Motivo
Sistema de archivos	Escritura en la carpeta del proyecto	Crear el proyecto y los documentos.
Red local	Puerto 8123	Servir el sitio para pruebas y capturas.
npm	Instalación de dependencias de desarrollo	docx, pdf-lib y puppeteer-core.
GitHub	repo, workflow	Crear repositorio público, subir código y habilitar Pages.
Chrome headless	Ejecución local	Capturas y conversión de HTML a PDF.

No se solicitó acceso a organizaciones, facturación ni configuraciones administrativas de la cuenta.

> **Sobre la publicación**
>
> GitHub Pages requiere un repositorio público en el plan gratuito. Publicar el proyecto expone el código y los datos simulados en Internet; es una decisión del equipo y debe confirmarse antes de ejecutar el comando de creación del repositorio. Si se prefiere mantenerlo privado, el sitio puede desplegarse en un repositorio privado con GitHub Pro o en un hosting alternativo.

5. Estructura del proyecto

El repositorio separa claramente los activos del sitio, los datos, la documentación y las herramientas de construcción. Esta organización facilita el mantenimiento y permite que GitHub Pages publique únicamente lo necesario.

```
plastrack/
├── index.html                # Dashboard (una sola página)
├── assets/
│   ├── css/styles.css        # Sistema de diseño y responsive
│   ├── js/
│   │   ├── app.js            # Controlador y estado
│   │   ├── analytics.js      # Filtros, KPIs y estadística
│   │   ├── charts.js         # Visualizaciones Chart.js
│   │   ├── map.js            # Capa Leaflet
│   │   └── format.js         # Formateo es-EC
│   └── img/                  # Favicon y portada social
├── data/
│   └── data_copernicus_sample.json
├── docs/
│   ├── Reporte_Tecnico_PlasTrack.pdf
│   ├── Reporte_Tecnico_PlasTrack.docx
│   ├── Memoria_de_Construccion.pdf
│   ├── Memoria_de_Construccion.docx
│   └── figuras/              # Diagramas y capturas
├── scripts/
│   ├── build-docs.mjs        # Genera PDF y DOCX
│   ├── serve.mjs             # Servidor local de pruebas
│   ├── content/              # Texto del reporte y la memoria
│   └── lib/                  # Renderizadores y estadística
├── tests/analytics.test.mjs  # Pruebas del núcleo analítico
├── .github/workflows/        # Verificación automática (opcional)
├── package.json
├── README.md
└── LICENSE
```

La carpeta scripts no se publica en el sitio, pero permanece en el repositorio para que la documentación pueda regenerarse. El archivo .nojekyll evita que GitHub procese el sitio con Jekyll. El archivo .gitignore excluye node_modules y los archivos temporales.

6. Proceso de construcción

El trabajo se ejecutó en ocho etapas. Cada una produjo un artefacto verificable y se cerró con una revisión antes de avanzar. Las etapas 1 a 7 se realizaron en una sola sesión de trabajo asistida por IA; la etapa 8 corresponde a la validación final y al despliegue.

6.1. Etapa 1. Lectura y descomposición de la consigna

Se extrajo el texto del PDF con una biblioteca de lectura (pdf-parse) y se identificaron los dos entregables —reporte tipo paper y dashboard en GitHub Pages— además de los cuatro criterios de la rúbrica. La consigna se convirtió en la matriz de requisitos de la Tabla 2. Esta etapa evitó omitir componentes exigidos, como la bitácora de prompts o el consumo del archivo JSON con nombre específico.

6.2. Etapa 2. Diseño del dataset

Se definieron 16 campos y 15 registros con estacionalidad coherente: nubosidad alta en febrero, descenso progresivo hasta junio y aptitud inversamente proporcional. Las coordenadas se ubicaron dentro de las islas y el área se ajustó al tamaño relativo de cada una. El archivo se guardó en data/data_copernicus_sample.json y se validó con una función que verifica campos obligatorios, rangos y formato de fecha.

6.3. Etapa 3. Arquitectura y sistema de diseño

Se optó por una arquitectura estática de cuatro capas y por un design system con variables CSS. Se definieron dos temas (oscuro y claro), una escala tipográfica, radios, sombras y una paleta con contraste suficiente. El diseño prioriza la lectura ejecutiva: hero con propuesta de valor, filtros persistentes, KPIs arriba, mapa y hallazgos en el centro, gráficos y tabla al final. La Figura 1 muestra el resultado en escritorio.

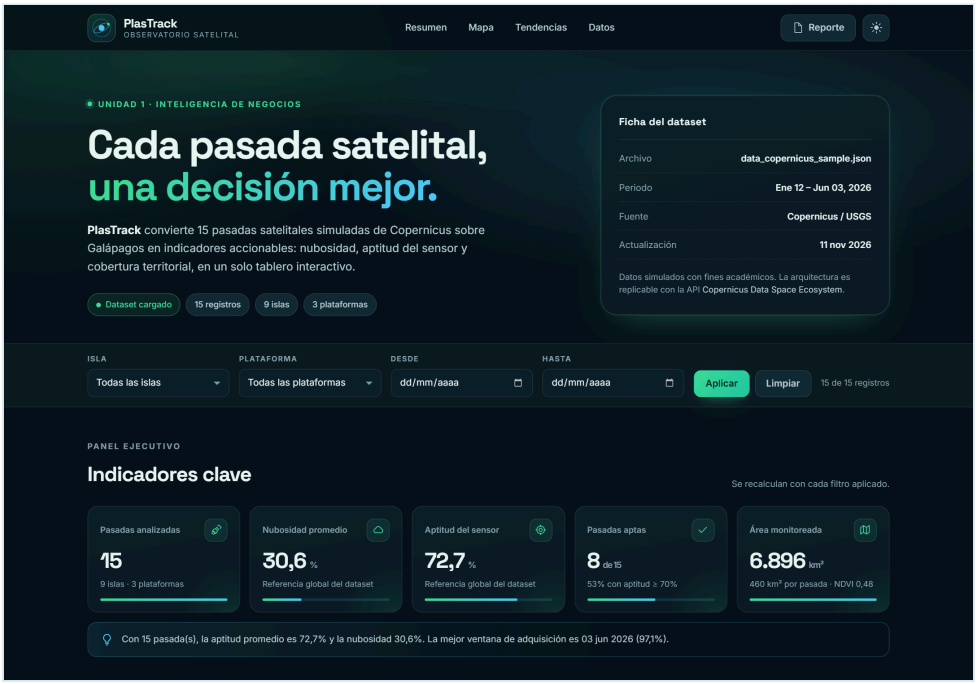


Figura 1. Vista del dashboard en escritorio (1440 px).

6.4. Etapa 4. Implementación de la lógica

La lógica se separó en cuatro módulos. analytics.js contiene funciones puras de filtrado, agregación, correlación y exportación; charts.js encapsula la creación y actualización de los cuatro gráficos; map.js gestiona Leaflet, los marcadores, el mapa de calor y las zonas; app.js orquesta el estado, los eventos y el renderizado. Esta separación permitió escribir pruebas unitarias sin navegador y reutilizar el mismo cálculo en los documentos generados.

6.5. Etapa 5. Interfaz y accesibilidad

Se implementaron filtros combinables, búsqueda con retardo, ordenamiento por columna, exportación CSV, cambio de tema persistente y control de capas del mapa. En accesibilidad se añadieron enlace de salto, etiquetas ARIA, roles de tabla, anuncios aria-live para los KPIs, foco visible, textos alternativos y respeto por prefers-reduced-motion. La auditoría de Lighthouse se usó como termómetro y se corrigieron los hallazgos reportados.

6.6. Etapa 6. Pruebas

Se escribieron pruebas automatizadas con node:test para filtros, medias, correlación, KPIs, validación de esquema, exportación CSV y clasificación por semáforo. Además se ejecutaron doce casos manuales en el navegador, documentados en el reporte. La Tabla 7 resume los resultados. Las pruebas se ejecutan con npm test y no requieren navegador.

6.7. Etapa 7. Documentación

El reporte y esta memoria se redactaron como contenido estructurado en JavaScript y se renderizan a HTML para producir PDF con Chrome headless y a DOCX con la biblioteca docx. Este enfoque garantiza que las cifras citadas se calculen desde el mismo dataset que alimenta el dashboard: si un registro cambia, los documentos se regeneran con los valores actualizados. Los scripts build-docs.mjs y build-figures.mjs automatizan el proceso.

6.8. Etapa 8. Validación y despliegue

Se verificó el sitio en Chrome y Edge, en resoluciones de 390, 768, 1024 y 1440 px, y se revisó la consola en busca de errores. Se generaron capturas de escritorio y móvil, se ejecutó Lighthouse y se comprobó la exportación CSV. Finalmente se inicializó el repositorio, se creó en GitHub y se activó Pages. El procedimiento exacto se detalla en la sección 9.

7. Decisiones técnicas

Tabla 5. Decisiones, alternativas y justificación

Decisión	Alternativa considerada	Justificación
Sin framework (vanilla JS)	React o Vue	El alcance es una página; un framework añadiría build, dependencias y peso sin beneficio claro.
Módulos ES nativos	Bundler con Webpack o Vite	Cero configuración, código legible y compatible con GitHub Pages.
JSON local	API de Copernicus en vivo	Reproducibilidad académica y funcionamiento sin credenciales ni cuotas.
GitHub Pages	Vercel, Netlify o servidor propio	Integración con Git, costo cero y URL estable.
Chart.js	D3.js o ECharts	Menor curva de aprendizaje y componentes suficientes para el alcance.
Leaflet + teselas Esri	Google Maps o Mapbox	Sin clave de API y con atribución simple; buena resolución en Galápagos.

Decisión	Alternativa considerada	Justificación
Diseño oscuro por defecto	Tema claro por defecto	Mayor impacto visual y coherencia con el dominio satelital; el tema claro está disponible.
Documentos generados por script	Redacción manual en Word	Consistencia de cifras y posibilidad de regenerar tras cambios en el dataset.

8. Cómo se trabajó con la IA

La sesión comenzó con la instrucción de revisar el PDF de la actividad, actuar como ingeniero de software, construir el proyecto en una carpeta local, indicar si se necesitaban permisos y generar un documento con la explicación del proceso. A partir de ahí se trabajó en ciclos cortos: analizar, proponer, implementar, ejecutar y corregir. La Tabla 6 resume las instrucciones principales y el resultado de cada una.

Tabla 6. Instrucciones principales de la sesión y resultados

#	Instrucción	Resultado
1	Revisar el PDF y extraer requisitos, entregables y rúbrica.	Requisitos identificados y matriz de trazabilidad.
2	Actuar como ingeniero de software y construir el proyecto completo.	Arquitectura, dataset y dashboard funcional.
3	Indicar si se necesitan permisos para la realización.	Análisis de permisos locales y de GitHub (sección 4.2).
4	Alojar el proyecto en una carpeta local.	Estructura creada y verificada.
5	Generar un documento con la explicación de cómo se realizó.	Esta memoria, en PDF y DOCX.
6	Preparar el reporte técnico con benchmarking, KPIs y bitácora.	Reporte tipo paper con tablas y anexos.
7	Verificar el dashboard, la responsividad y la accesibilidad.	Pruebas, capturas y auditoría Lighthouse.

El trabajo con IA incluyó correcciones. Por ejemplo, la primera versión del mapa usaba teselas oscuras de CARTO y los marcadores se perdían sobre el fondo; se cambió a imagen satelital de Esri y se aumentó el tamaño de los marcadores. También se detectó que el cálculo del encabezado mostraba fechas en formato ISO y decimales con punto; se corrigió con las utilidades de formato es-EC. Estos ajustes se registraron en la bitácora del reporte.

9. Pruebas y aseguramiento de calidad

9.1. Pruebas automatizadas

Las pruebas unitarias cubren el núcleo analítico y se ejecutan con Node.js. No dependen del DOM ni de librerías externas, lo que las hace rápidas y estables. La Tabla 7 lista los grupos de casos.

Tabla 7. Cobertura de las pruebas automatizadas

Grupo	Casos	Qué verifica
Filtrado	3	Isla, plataforma y rango de fechas, incluidos casos vacíos.
Estadística	2	Media y correlación de Pearson con casos límite.
KPIs	3	Totales, tasa de aptas y área acumulada.
Semáforo	1	Clasificación correcta en los tres umbrales.
Exportación	1	Encabezados, número de filas y escape de caracteres.
Validación	2	Esquema válido y detección de campos faltantes.

9.2. Pruebas manuales

Se ejecutaron doce casos manuales en el navegador: filtros simples y combinados, limpieza, búsqueda, ordenamiento, exportación, tema, capas del mapa, navegación por teclado, vista móvil y carga sin teselas. Todos finalizaron correctamente. El detalle se encuentra en la Tabla 13 del reporte técnico.

9.3. Accesibilidad y rendimiento

Tabla 8. Resultados de Lighthouse

Categoría	Puntaje	Observación
Accesibilidad	100/100	Se corrigieron el nombre accesible del enlace al reporte y se agruparon los marcadores del mapa.
Buenas prácticas	100/100	Sin errores de seguridad ni de consola.
SEO	100/100	Metadatos, título y descripción presentes.

El sitio carga sin frameworks y con un dataset de menos de 8 KB. La verificación se realizó con Chrome headless a 1440 px y con emulación móvil a 390 px. La Figura 2 muestra la vista móvil, donde se comprobó que no existe desplazamiento horizontal de la página y que la tabla se desplaza dentro de su contenedor.



Figura 2. Vista móvil del dashboard (390 px).

10. Despliegue en GitHub Pages

El despliegue se realiza una sola vez y luego se actualiza con cada push. Los comandos siguientes asumen que el repositorio local ya existe y que GitHub CLI está autenticado.

```
# 1. Verificar la sesión de GitHub
gh auth status

# 2. Inicializar el repositorio local (solo la primera vez)
git init
git add .
git commit -m "feat: dashboard PlasTrack con KPIs, mapa y gráficos"

# 3. Crear el repositorio público y subir el código
gh repo create plastrack --public --source=. --remote=origin --push

# 4. Activar GitHub Pages con despliegue desde Actions
gh api -X POST repos/{owner}/plastrack/pages -f build_type=workflow

# 5. Verificar la publicación
gh api repos/{owner}/plastrack/pages --jq '.html_url'
```

La URL resultante es https://carlos00012.github.io/plastrack/. GitHub tarda entre uno y dos minutos en construir el sitio la primera vez. Si se actualiza el contenido, basta con hacer commit y push; el flujo de GitHub Actions vuelve a publicar automáticamente. El archivo .nojekyll evita que los archivos que comienzan con guion bajo sean ignorados.

> **Antes de publicar**
>
> Confirmar que el repositorio será público, que los nombres de los integrantes están completos en la portada y que la URL se añadió al README y al anexo D del reporte. Revisar que no existan credenciales ni archivos personales en el historial de Git.

11. Mantenimiento y actualización

Para añadir una nueva pasada basta con agregar un objeto al arreglo JSON con los mismos campos y volver a cargar el sitio. La validación mostrará una advertencia en consola si falta un campo o si un valor está fuera de rango, pero el tablero seguirá funcionando con los registros válidos. Si se modifican los umbrales de aptitud, deben actualizarse las constantes APTITUDE_THRESHOLD y EXCELLENT_THRESHOLD en analytics.js y la ficha de KPIs del reporte.

Para regenerar los documentos después de cambiar el dataset se ejecuta npm run build:docs. El script recalcula las cifras, reconstruye las tablas y produce nuevamente los archivos PDF y DOCX. Las capturas se actualizan con npm run build:figures, que requiere Chrome instalado y el servidor local en ejecución.

Tabla 9. Tareas de mantenimiento

Tarea	Frecuencia	Procedimiento
Añadir registros	Según disponibilidad	Editar el JSON y validar el esquema.
Actualizar capturas	Al cambiar el diseño	npm run serve + npm run build:figures.
Regenerar documentos	Al cambiar datos o texto	npm run build:docs.
Ejecutar pruebas	Antes de cada push	npm test.
Revisar dependencias	Mensual	npm outdated y actualización controlada.

12. Riesgos y limitaciones

Tabla 10. Riesgos identificados y mitigación

Riesgo	Impacto	Mitigación
Dependencia de CDN y teselas	Medio	Los gráficos y KPIs funcionan sin red; se documenta la limitación y se propone empaquetado local.
Datos simulados	Alto para conclusiones ambientales	Se declara el carácter sintético en el sitio y en los documentos.
Muestra pequeña por isla	Medio	Se advierte que las comparaciones por isla no son concluyentes.
Cambios de precios de licencias	Bajo	Se usan rangos y se indica la fecha de consulta.
Pérdida de la sesión de GitHub	Bajo	El repositorio local conserva el historial; se puede volver a publicar.
Accesibilidad incompleta	Medio	Auditoría Lighthouse y correcciones; se recomienda prueba con lector de pantalla.

13. Guion de exposición y defensa

La rúbrica individual evalúa la exposición oral y la defensa técnica. Se propone la siguiente distribución de siete minutos en total, adaptable al número de miembros del equipo.

Tabla 11. Guion sugerido de exposición

Bloque	Tiempo	Contenido clave
Problema y contexto	1 min	Galápagos, Copernicus, nubosidad y necesidad de decidir con datos.
Benchmarking	1,5 min	Criterios, pesos, resultados y por qué ganó el stack web.
Arquitectura y KPIs	1,5 min	Cuatro capas, flujo del JSON y ficha de indicadores.
Demo del dashboard	2 min	Filtros, mapa, gráficos, tabla y exportación.
Resultados y conclusiones	1 min	Nubosidad 30,6%, aptitud 72,7%, ventana de junio y aprendizajes.

Preguntas frecuentes. ¿Por qué no Power BI? Porque el requisito de mapa con control total y URL pública sin licencias favorece el stack web. ¿Los datos son reales? No, son simulados y se declaran como tales; la arquitectura es replicable con la API de Copernicus. ¿Cómo se calcula la aptitud? Como porcentaje de píxeles válidos tras el enmascaramiento, con una relación inversa a la nubosidad en el dataset simulado. ¿Qué pasa si falla el CDN? Los KPIs, la tabla y los gráficos de datos siguen funcionando; solo se pierde la imagen de fondo del mapa. ¿Cómo se garantiza la calidad? Con pruebas unitarias, doce casos manuales, auditoría Lighthouse y revisión de código.

14. Checklist de la rúbrica

Tabla 12. Correspondencia con los criterios de evaluación

Criterio (peso)	Evidencia	Estado
Reporte técnico y benchmarking (2,5)	Reporte con Tablas 1–4, arquitectura, ficha de KPIs y bitácora de 13 prompts.	Completo
Dashboard en GitHub Pages (2,5)	Sitio funcional, responsivo, con KPIs, Leaflet, Chart.js y consumo del JSON.	Completo
Exposición oral (2,5)	Guion de la Tabla 11 y demo en vivo del tablero.	Preparado
Defensa técnica (2,5)	Preguntas frecuentes, arquitectura documentada y código comentado.	Preparado

Antes de la entrega se recomienda verificar: (i) que la URL de Pages esté activa y sin errores en consola; (ii) que la portada tenga los nombres correctos; (iii) que el PDF y el DOCX abran sin advertencias; (iv) que el repositorio tenga README, licencia y .gitignore; (v) que el dataset conserve el nombre data_copernicus_sample.json; y (vi) que la bitácora de prompts esté incluida en el reporte.